EXERCISE 2 :

SCENARIO 1:

Write a stored procedure `SafeTransferFunds` that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
)
AS
    v_balance Accounts.Balance%TYPE;
BEGIN

    -- Get balance of source account
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    -- Check whether sufficient balance is available
    IF v_balance >= p_amount THEN

        -- Deduct amount from source account
        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        -- Add amount to destination account
        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Fund Transfer Successful');

    ELSE
```

```
        RAISE_APPLICATION_ERROR(-20001,'Insufficient Balance');

    END IF;

EXCEPTION

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
```

OUTPUT:

BEFORE:

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED
1	1	John Doe	5/15/1985, 12:00:0(	1000	6/28/2026, 6:25:43
2	2	Jane Smith	7/20/1990, 12:00:0(	1500	6/28/2026, 6:25:43
3	3	Robert King	2/10/1955, 12:00:0(	12000	6/28/2026, 4:06:41
4	4	Mary Thomas	9/18/1960, 12:00:0(	18000	6/28/2026, 4:06:41
5	5	David Wilson	12/25/1998, 12:00:(	9000	6/28/2026, 4:06:41

AFTER:

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	1	Savings	0	6/28/2026, 6:25:43
2	2	2	Checking	2500	6/28/2026, 6:25:43
3	3	3	Savings	12000	6/28/2026, 4:06:41
4	4	4	Checking	18000	6/28/2026, 4:06:41
5	5	5	Savings	9000	6/28/2026, 4:06:41

SCENARIO 2:

Write a stored procedure `UpdateSalary` that increases the salary of an employee by a given percentage. If the Employee ID does not exist, handle the exception and log an error message.

QUERY:

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_employeeid IN NUMBER,
    p_percentage IN NUMBER
)
AS
    v_salary Employees.Salary%TYPE;
BEGIN

    -- Check if employee exists and get current salary
    SELECT Salary
    INTO v_salary
    FROM Employees
    WHERE EmployeeID = p_employeeid;

    -- Update salary
    UPDATE Employees
    SET Salary = Salary + (Salary * p_percentage / 100)
    WHERE EmployeeID = p_employeeid;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID does not exist.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
```

OUTPUT:

```
BEGIN
    UpdateSalary(1,10);
END;
/
```

	EMPLOYEEID	NAME	POSITION	SALARY	DEPARTMENT	HIREDATE
1	1	Alice Johnson	Manager	102487	HR	6/15/2015, 12:00:0(
2	2	Bob Brown	Developer	60000	IT	3/20/2017, 12:00:0(

```
BEGIN
    UpdateSalary(100,10);
END;
/
```

```
SQL> BEGIN
         UpdateSalary(100,10);
     END;


Employee ID does not exist.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.004
```

SCENARIO 3:

Write a stored procedure `AddNewCustomer` that inserts a new customer into the `Customers` table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

QUERY:

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_customerid IN NUMBER,
    p_name IN VARCHAR2,
    p_dob IN DATE,
    p_balance IN NUMBER
)
AS
BEGIN

    INSERT INTO Customers
    (CustomerID, Name, DOB, Balance, LastModified)
    VALUES
    (p_customerid, p_name, p_dob, p_balance, SYSDATE);

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Added Successfully.');

EXCEPTION

    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Customer ID already exists.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
```

OUTPUT:

BEFORE:

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	5/15/1985, 12:00:0(	1000	6/28/2026, 6:25:43	FALSE
2	2	Jane Smith	7/20/1990, 12:00:0(	1500	6/28/2026, 6:25:43	FALSE
3	3	Robert King	2/10/1955, 12:00:0(	12000	6/28/2026, 4:06:41	TRUE
4	4	Mary Thomas	9/18/1960, 12:00:0(	18000	6/28/2026, 4:06:41	TRUE
5	5	David Wilson	12/25/1998, 12:00:(	9000	6/28/2026, 4:06:41	FALSE

AFTER:

```
BEGIN
    AddNewCustomer(
        3,
        'Robert King',
        TO_DATE('1995-10-20','YYYY-MM-DD'),
        5000
    );
END;
```

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	5/15/1985, 12:00:0(	1000	6/28/2026, 6:25:43	FALSE
2	2	Jane Smith	7/20/1990, 12:00:0(	1500	6/28/2026, 6:25:43	FALSE
3	3	Robert King	2/10/1955, 12:00:0(	12000	6/28/2026, 4:06:41	TRUE
4	4	Mary Thomas	9/18/1960, 12:00:0(	18000	6/28/2026, 4:06:41	TRUE
5	5	David Wilson	12/25/1998, 12:00:(	9000	6/28/2026, 4:06:41	FALSE